

วันที่ 4

Scheduling เชิงลึก และ Employee Rostering

predicate · Flexible Job Shop · Rostering · regular constraint

predicate

นิยาม constraint ใหม่

Flexible

เลือกเครื่องได้

Rostering

3 กะ + กฎ

regular

DFA pattern

กำหนดการวันที่ 4

09:00–09:45

**predicate**นิยาม *constraint* ใหม่ — *no_overlap*, *disjunctive* เทียบกัน

09:45–11:00

**Flexible Job Shop**เลือกเครื่องจักรได้ — *machine variable*, *duration* เปลี่ยนตาม

11:00–12:00

**Workshop 4A***Flexible Manufacturing* — 4 orders, 3 tasks, 3 machines

13:00–14:30

**Employee Rostering**3 กะ, *demand*, *rest rule*, *fairness* — ปัญหาจริงในโรงงาน

14:30–15:30

**regular constraint***DFA* → *transition table* → บังคับ *shift pattern* ซับซ้อน

15:30–16:00

**Workshop 4B***Shift Scheduling* — 10 คน, 14 วัน, กฎแรงงาน

predicate — นิยาม Constraint ใหม่

`predicate <name>(<args>) = <bool-expression>;` → ใช้แทน constraint ได้ทุกที่

% นิยาม predicate: 2 task ห้าม overlap

```
predicate no_overlap(var int: s1, int: d1, var int: s2, int: d2) =
  s1 + d1 <= s2  \ /  s2 + d2 <= s1;
```

% ใช้งาน — อ่านเข้าใจง่ายกว่า inline

```
constraint no_overlap(start[A], dur[A], start[B], dur[B]);
```

ทำไมต้องใช้ predicate:

 อ่านง่ายขึ้น

`no_overlap(...)` ชัดกว่า `s1+d1<=s2 \ / s2+d2<=s1`

 นามานำมาใช้ซ้ำได้

เรียกใช้ได้ทุกที่ในโมเดล — ไม่ต้องเขียนซ้ำ

 modular

ซ่อนรายละเอียด — เปลี่ยน implementation ได้โดยไม่กระทบส่วนอื่น

predicate no_overlap vs. disjunctive

predicate (pairwise)

```
predicate no_overlap(...) =
  s1+d1<=s2 \/ s2+d2<=s1;

constraint forall(t in TASK)(
  forall(j,k in JOB where j<k)(
    no_overlap
    (s[j,t],d[j,t],s[k,t],d[k,t]));
```

% $n*(n-1)/2$ constraints ต่อ task

% ยึดหยุ่น – ใส่ extra condition ได้

disjunctive (global)

```
include "disjunctive.mzn";

constraint forall(t in TASK)(
  disjunctive
  ([s[j,t]|j in JOB],
   [d[j,t]|j in JOB]));
```

% 1 constraint ต่อ task

% solver มี algorithm พิเศษ

% เร็วกว่าสำหรับปัญหาใหญ่

Flexible Job Shop — เลือกเครื่องจักรได้

Classic Job Shop: task ใช้เครื่องตายตัว → Flexible: task เลือกเครื่องได้ duration ต่างกัน

	Classic Job Shop	Flexible Job Shop
เครื่องที่ใช้	Fixed ต่อ task	เลือกได้ — เป็น decision variable
Duration	$d[\text{job}, \text{task}]$	$d[\text{job}, \text{task}, \text{machine}]$ — ขึ้นกับเครื่อง
ตัวแปรเพิ่ม	—	$\text{machine}[\text{job}, \text{task}]: \text{var MACHINE}$
Constraint เพิ่ม	—	$\text{dur}[j, t, \text{machine}[j, t]] > 0$
Complexity	ต่ำกว่า	สูงกว่า — solver ต้องเลือกเครื่องด้วย

ประยุกต์จริง: เครื่องจักรทดแทน, ขนาดงานกับขนาดเครื่อง, ค่าใช้จ่ายต่างกันต่อเครื่อง

Flexible Job Shop — โมเดล

```
enum JOB; enum TASK; enum MACHINE;

array[JOB,TASK,MACHINE] of int: dur;
% 0 = เครื่องนั้นทำ task นี้ไม่ได้

% ตัวแปรสำคัญ: เลือกเครื่อง!
array[JOB,TASK] of var MACHINE: machine;
array[JOB,TASK] of var 0..total: start;

% 1. เลือกเฉพาะเครื่องที่ทำได้
constraint forall(j,t)(dur[j,t,machine[j,t]] > 0);

% 2. Precedence ใน job
% start[j,t]+dur[j,t,machine[j,t]] <= start[j,t+1]

% 3. No-overlap: เครื่องเดียวกันทีละ 1
% machine[j,t]=m /\ machine[k,t]=m -> no_overlap
```

◀ 3D array!
indexed by job, task, machine

◀ machine เป็น variable
— Solver เลือกให้

◀ duration ขึ้นกับ
เครื่องที่เลือก

◀ no-overlap: เช็คเฉพาะ
cases ที่เครื่องตรงกัน



Workshop 4A — Flexible Manufacturing

เวลา: 60 นาที | ไฟล์: `workshop4a_starter.mzn` + `workshop4a_data.dzn`

โจทย์: 4 ออร์เดอร์ (J1-J4), 3 ขั้นตอน (T1-T3), 3 เครื่องจักร (MA,MB,MC)
แต่ละ task ทำได้บางเครื่องเท่านั้น (0 = ทำไม่ได้)

1 เพิ่ม constraint เลือกเครื่องที่ทำได้:

2 เพิ่ม Precedence:

3 เพิ่ม No-overlap ต่อเครื่อง:

4 solve minimize makespan:

5 (ท้าทาย) เปรียบเทียบ makespan:

ลบ constraint เลือกเครื่อง → solver เลือกไม่ดี → makespan นานขึ้น?

ช่วงบ่าย

Employee Rostering และ regular constraint

demand: ทุกวันต้องมีคนครบทุกกะ

rest rule: หลังกะตึกต้องหยุด

fairness: กระจาย shift อย่างเป็นธรรม

regular: DFA บังคับ pattern ที่ซับซ้อน

Employee Rostering — ปัญหาจริงในโรงงาน

Rostering = กำหนดว่าใครทำอะไรวันไหน โดยตรงทุก constraint พร้อมกัน



Demand Constraints

- ▶ กะเช้าต้องมี $\geq \text{req_morning}$ คน
- ▶ กะดึกต้องมี $\geq \text{req_night}$ คน
- ▶ ทุกวันทุกกะต้องครบ



Rest Rules

- ▶ หลังกะดึกต้องหยุดอย่างน้อย 1 วัน
- ▶ ห้ามทำ 3 กะติดติดต่อกัน
- ▶ ทำงานติดต่อกันไม่เกิน 6 วัน



Fairness

- ▶ กะดึกกระจายเท่ากัน
- ▶ วันหยุดกระจายเท่ากัน
- ▶ Minimize กะดึกรวม (หรือ max-min)

MiniZinc: `array[EMPLOYEE, DAY] of var SHIFT: roster;` — 1 ตัวแปรต่อ 1 slot

Rostering — โมเดล MiniZinc

```
enum SHIFT = { M, E, N, O }; % morning/eve/night/off

array[EMP, DAY] of var SHIFT: roster;

% Demand
constraint forall(d in DAY)(
  sum(e in EMP)(roster[e,d]=M) >= req_M
  /\ sum(e in EMP)(roster[e,d]=N) >= req_N);

% Rest after night
constraint forall(e,d where d < max(DAY))(
  roster[e,d]=N -> roster[e,enum_next(DAY,d)]=0);

% Fairness: minimize กะดึกของทุกคน
var int: nights = max(e)(sum(d)(bool2int(roster[e,d]=N)));

solve minimize nights;
```

◀ ตัวแปรหลัก:
1 cell ต่อ (พนักงาน,วัน)

◀ Demand:
bool2int auto-coerce

◀ Rest rule:
enum_next สำหรับวันถัดไป

◀ Minimize nights:
เพื่อความเป็นธรรม

regular constraint — Pattern ที่ซับซ้อน

```
regular(sequence, transition_table, start_state, accepting_states)
```

เมื่อไรต้องใช้ regular:

constraint ธรรมดาเขียนยาก

ห้ามทำตึก 3 คั่นติดกัน: ต้องจำ 'นับกี่คั่นแล้ว'

กฎขึ้นกับ history

ทำงาน 5 วัน → ต้องหยุด 2 วัน → ทำได้อีก...

กฎเป็น sequence

เฉพาะลำดับกะที่กฎหมายแรงงานอนุญาต

หลักการ: DFA (Deterministic Finite Automaton)

1. วาด state diagram: แต่ละ state = 'ประวัติที่เกี่ยวข้อง' (เช่น กะตึกติดกันกี่คั่น)
2. กำหนด transitions: จาก state ไหน รับ input อะไรไป state ไหน (<=> illegal)
3. แปลงเป็น transition table 2D array → ใส่ใน regular()

ออกแบบ DFA สำหรับกฎกะ

กฎ: ห้ามกะดึก (N) 3 ค่ำติดกัน / ในทุก 4 วัน ต้องมีหยุด (O) อย่างน้อย 1 วัน

S1

เริ่ม/หยุด

S2

ทำงาน 1 วัน

S3

ทำงาน 2 วัน

S4

ทำงาน 3 วัน

S5

ดึก 1 ค่ำ

S6

ดึก 2 ค่ำ

Transition Table:

State	d (day)	n (night)	o (off)
S1	S2	S3	S1
S2	S4	S4	S1
S3	S4	S5	S1
S4	S6	S6	S1
S5	S6	<> ← ห้าม!	S1
S6	<> ← ห้าม!	<> ← ห้าม!	S1

regular constraint — โค้ด MiniZinc

```
include "regular.mzn";  enum SHIFT = { d, n, o };  enum STATE = { S1..S6 };

% Transition table [state, shift] -> next_state (<> = illegal)

array[STATE,SHIFT] of opt STATE: tr =
  [|
    d:  n:  o:
    | S1:  S2, S3, S1 | S2:  S4, S4, S1 | S3:  S4, S5, S1
    | S4:  S6, S6, S1
    | S5:  S6, <>, S1   % ห้ามติด 3 คั่นติดต่อกัน!
    | S6:  <>, <>, S1 |]; % ต้องหยุดก่อน

% บังคับทุกพนักงาน

constraint forall(e in NURSE)(regular([roster[e,d]|d in DAY], tr, S1, STATE));
```

<> = option type 'absent' — ใน MiniZinc หมายถึง illegal transition ทำให้ constraint fail



Workshop 4B — Shift Scheduling โรงงาน 3 กะ

เวลา: 30 นาที | ไฟล์: workshop4b_starter.mzn

โจทย์: พนักงาน 10 คน, 14 วัน — กะ M/E/N/O

Demand: เช้า $\geq$ 4, บ่าย $\geq$ 3, ดึก $\geq$ 2 | Minimize กะดึกรวม

- 1 **เพิ่ม demand constraint:**
- 2 **เพิ่ม rest after night:**
- 3 **เพิ่ม max 6 วันทำงานติด:**
- 4 **solve minimize total_nights:**
- 5 **(ท้าทาย) ใช้ regular:** เพิ่ม regular constraint สำหรับ rule ห้ามดึก 3 ค่ำติด

สรุปวันที่ 4 — สิ่งที่ต้องจำ



predicate — นิยาม constraint ที่นำมาใช้ซ้ำได้

predicate no_overlap(s1,d1,s2,d2) = ... — อ่านง่าย reusable ซ่อน complexity



Flexible Job Shop — เพิ่มมิติการเลือกเครื่อง

machine[j,t]: var MACHINE — dur[j,t,machine[j,t]] — constraint เลือกเฉพาะที่ทำได้



Rostering = demand + rest + fairness

sum(e)(roster[e,d]=N) >= req_N | N → 0 rest rule | minimize nights



regular — DFA บังคับ shift pattern

วาด DFA ก่อน → transition table → regular([roster[e,d] | d in DAY], tr, S1, STATE)

Quick Reference: predicate และ function

```
% — PREDICATE (returns bool) —  
  
predicate no_overlap(var int: s1, int: d1, var int: s2, int: d2) =  
    s1 + d1 <= s2  \/\  s2 + d2 <= s1;  
  
% — FUNCTION (returns value) —  
  
function var int: total_dur(var MACHINE: m, JOB: j, TASK: t) =  
    dur[j, t, m];  
  
% — TEST (fixed bool, สำหรับ parameter เท่านั้น) —  
  
test is_weekday(int: d) = d mod 7 <= 5;  
  
% — LOCAL VARIABLE ใน predicate —  
  
predicate overlap_free(...) = let {  
    var int: gap = s2 - s1 - d1;  
} in gap >= 0;
```


Quick Reference: regular constraint

```
include "regular.mzn";

% Signature

regular(array[int] of var $$Val: x,
        array[$$State,$$Val] of opt $$State: d,
        $$State: q0,
        set of $$State: F)

% — ตัวอย่าง: ห้าม 3 วันติดกัน —

enum STATE = { S0, S1, S2 };

array[STATE,SHIFT] of opt STATE: tr =
  [| S0: S1, S0    % d=S1, o=S0
   | S1: S2, S0    % d=S2, o=S0
   | S2: <>, S0 |]; % illegal, o=S0

constraint forall(e in EMP)(
  regular([roster[e,d]|d in DAYS], tr, S0, STATE));
```

ข้อผิดพลาดที่พบบ่อย — วันที่ 4

- ✗ `predicate f(var int: x) = let {var int: y} in x=2*y;`
- ✓ `predicate f(var int: x) = let {var int: y = x div 2} in x=2*y;`

local variable ใน predicate ต้องถูก define ถ้าอยู่ใน negative context

- ✗ `dur[j,t,machine[j,t]] % ก่อนเพิ่ม constraint dur>0`
- ✓ `constraint forall(j,t)(dur[j,t,machine[j,t]] > 0);`

Flexible Job Shop: ต้องมี constraint บังคับว่าเลือกเฉพาะเครื่องที่ทำได้

- ✗ `roster[e, d+1] % ถ้า DAY เป็น enum`
- ✓ `roster[e, enum_next(DAY, d)] % ใช้ enum_next`

ถ้า index เป็น enum ต้องใช้ enum_next ไม่ใช่ +1

- ✗ `regular(roster[e], ...) % roster[e] เป็น 2D array`
- ✓ `regular([roster[e,d] | d in DAY], ...) % ต้องสร้าง 1D sequence`

regular รับ 1D array — ต้องสร้าง sequence ของแต่ละพนักงานก่อน

Day 5 Preview

Vehicle Routing และ Bin Packing

TSP — circuit() global constraint

VRP — หลาย vehicle, หลายลูกค้า, capacity จำกัด

Bin Packing — minimize จำนวนกล่อง

Knapsack — set variables (var set of ITEMS)

กรณีศึกษา: ผลลัพธ์ Rostering จริง

% ตาราง 7 พนักงาน 10 วัน (M=เช้า, N=ดึก, o=หยุด)

Nurse(1): d o n o n o d d n o

Nurse(2): d o d n n o d d n o

% กะดึก: 3 วัน/คน เป็นธรรมชาติ

Key Observations:

Pattern N→O ถูกบังคับ

หลังกะดึกทุกครั้งจะมี o ตามมาเสมอ — *regular* ทำงาน

กะดึกกระจาย

minimize total_nights ทำให้ solver กระจาย N ออกไป

Trade-off

minimize nights อาจไม่ *minimize total_cost* — ต้องเลือก *objective* ให้ตรงความต้องการ

ปัญหาจริงซับซ้อนกว่า

วันลา, ทักษะพิเศษ, ข้อจำกัดส่วนตัว → เพิ่ม *constraint* ได้เรื่อยๆ

Quick Reference: Flexible + Rostering Patterns

```
% — FLEXIBLE JOB SHOP —————
array[JOB,TASK] of var MACHINE: machine;
constraint forall(j,t)(dur[j,t,machine[j,t]] > 0);

% Precedence: start[j,t]+dur[j,t,machine[j,t]] <= start[j,t+1]

% — ROSTERING STRUCTURE —————
enum SHIFT = { M, E, N, O };
array[EMP,DAY] of var SHIFT: roster;
% Demand: sum(e)(roster[e,d]=M) >= req_M
% Rest:   roster[e,d]=N -> roster[e,enum_next(DAY,d)]=0
% Fair:   solve minimize sum(e,d)(bool2int(roster[e,d]=N))

% — REGULAR FOR PATTERNS —————
regular([roster[e,d]|d in DAY], tr, S0, STATE)
% tr = transition table | S0 = start state
% <> in tr = illegal transition (option type)
```